LLM-OS v3

Intelligent Hybrid Operating System

Project Report

Group Members

#	Name	Roll No
01	Muhammad Talha	24K-0834
02	Muhammad Irfan	24K-0740
03	Muhammad Rameel	24K-0758

Instructor	Mr. Ubaidullah
Submission Date	11 / 05 / 2026
Course	Operating Systems
Program	BS Computer Science

1. Summary

LLM-OS v3 is a sophisticated hybrid operating system that combines a full-featured C++ kernel with an LLM-powered agentic AI layer. The project demonstrates core OS concepts including process management, CPU scheduling with priority and round-robin preemption, virtual memory with four page-replacement policies, semaphore-based synchronization with deadlock detection, and a complete system-call interface — while integrating modern AI capabilities for intelligent task execution and long-term knowledge retention.

2. Introduction

Modern computing environments demand kernels that efficiently manage competing processes, memory pressure, and concurrent access to shared resources. LLM-OS v3 addresses these challenges while simultaneously acting as a research platform for AI-enhanced operating systems. By embedding a ReAct-based language model agent directly above the kernel, the system can reason about its own state using natural language, store insights across sessions via ChromaDB, and expose a rich web interface for multi-user interaction.

The OS side of the project covers every major topic from the Operating Systems curriculum: process-control blocks, priority scheduling with starvation prevention, virtual memory paging with LRU/FIFO/CLOCK/LFU eviction, POSIX-style semaphores, deadlock detection via resource-allocation graph cycle detection, and a TCP-based system-call RPC interface.

Technology Stack

Layer	Technology
Kernel (core OS logic)	C++17 — POSIX threads, TCP sockets, STL
Kernel Bridge	Python 3 — dual-mode: live C++ or simulation
AI Agent	Python — OpenRouter API, ChromaDB, sentence-transformers
Web Interface	Flask + HTML/JS
Build System	GNU Make
Operating System Target	Linux / Ubuntu

3. Project Description

The system is structured as four tightly integrated layers. At the base sits the C++ kernel that owns all hardware-level abstractions. Above it the Python Kernel Bridge translates high-level Python calls into TCP syscall packets (or, when the C++ binary is absent, emulates the entire kernel in Python). The LLM Agent layer consumes kernel state and external APIs to perform intelligent tasks. Finally, the Flask web interface exposes everything to end-users through a browser-based chat and dashboard.

3.1 Process Management Module (kernel/pcb.h)

Each process is represented by a Process Control Block (PCB) containing its PID, username (owner), state, priority, aging ticks, turn count, page-fault counter, pages-swapped count, creation / last-active timestamps, and a rolling 10-entry syscall log. The five legal process states are:

- NEW
- READY
- RUNNING

- WAITING
- TERMINATED

Effective Priority is computed as *priority - (aging_ticks / 5)*, automatically boosting long-waiting processes to prevent starvation. A lower effective priority value means higher scheduling precedence (range 0–10).

The **ProcessTable** class wraps a mutex-protected *std::map<int,PCB>*, providing thread-safe process creation, state transition, aging, and ready-queue extraction sorted by effective priority.

3.2 CPU Scheduling Module (kernel/scheduler.h)

The Scheduler implements **Priority + Round-Robin preemptive scheduling** with a configurable time quantum (default 500 ms). Three background threads run concurrently inside the kernel:

Thread	Role
scheduler_tick	Fires every quantum; ages READY processes; logs effective-priority queue every 10 ticks
memory_monitor	Polls RAM utilisation every 10 s; emits warnings at 70 % and 90 % occupancy
stats_printer	Dumps process-table and memory statistics every 30 s

All threads share a single *std::atomic<bool> stop_all_* flag and join cleanly on shutdown. This multi-threaded design mirrors real kernel architecture where the scheduler, memory manager, and statistics subsystems operate concurrently.

3.3 Virtual Memory Management (kernel/page_table.h)

The PageTable manages a fixed-size RAM (default 10 frames). Each PageEntry stores the page ID, frame ID, valid/dirty/reference bits, last-access timestamp, frequency counter, and content string. Four page-replacement policies are fully implemented:

Policy	Mechanism
LRU (Least Recently Used)	Doubly-linked list + position map; O(1) move-to-front on access; evict from tail
FIFO (First-In First-Out)	std::queue; evicts the oldest resident page; minimal overhead
CLOCK (Second Chance)	Circular ring with a clock hand; clears reference bit on first pass, evicts on second
LFU (Least Frequently Used)	Scans all valid entries; evicts minimum frequency, using last-access time as tiebreaker

Evicted pages are sent to VectorDB (secondary storage) with semantic-embedding support, enabling similarity-based page recall. The policy can be switched at runtime via *set_policy()*, which rebuilds the internal data structures without clearing pages.

3.4 Process Synchronization (kernel/semaphore.h)

The SemaphoreManager provides named binary and counting semaphores. Core operations:

- **create(name, init_value)** — Instantiates a new semaphore; rejects duplicates
- **wait(name, pid)** — Decrements value if > 0 and records assignment edge; otherwise enqueues PID, records request edge, and triggers deadlock check
- **signal(name, pid)** — Removes assignment edge; wakes the next waiter from the FIFO queue, or increments value if queue empty

Deadlock Detection: The manager maintains an assignment graph (semaphore → process) and a request graph (process → semaphore). On every blocking *wait()* call it constructs a waits-for graph and runs DFS cycle detection. When a cycle is found the affected PIDs and a preemption suggestion are printed immediately.

3.5 System Call Interface (kernel/syscall.h)

14 system calls are implemented, grouped into four categories. Each returns a JSON-formatted result containing a status flag, response data, and an error message where applicable.

Category	System Calls
Process	proc_fork, proc_run, proc_wait, proc_ready, proc_exit, proc_status
Memory	mem_alloc, mem_alloc_full, mem_read, mem_free
Semaphore	sem_create, sem_wait, sem_signal, sem_destroy
File I/O	fs_read, fs_write, fs_delete

Syscalls are dispatched over a TCP socket on port 9000 using JSON serialisation. This clean IPC boundary allows the Python Kernel Bridge to forward calls to the native binary or fall back to a pure-Python simulation with zero code changes in upper layers.

3.6 VectorDB — Secondary Storage (kernel/vector_db.h)

VectorDB acts as the disk tier of the memory hierarchy. Evicted pages are serialised to vector embeddings using the TextEmbedder class (vocabulary construction + cosine similarity scoring). This design is intentionally novel: it means that semantically related pages can be located efficiently, and it demonstrates how modern AI data structures can serve classical OS roles.

3.7 Python Kernel Bridge (kernel_bridge.py)

The bridge operates in two modes. In **Live Mode** it connects to the running C++ kernel_server via TCP and proxies every syscall. In **Simulation Mode** (automatic fallback when the binary is absent) it emulates all OS behaviour in Python — including LRU eviction, process table management, and scheduler tick logic — providing full functionality without compilation.

4. Methodology

Phase	Description
Requirements Analysis	Mapped OS curriculum topics to implementable components; identified C++ / Python boundary
System Design	Defined four-layer architecture; chose TCP/JSON as IPC protocol; selected four page-replacement algorithms
Kernel Implementation	Developed C++17 headers for PCB, PageTable, Semaphore, Scheduler, Syscall, VectorDB; compiled with pthread and O2
Bridge & Simulation	Built Python kernel_bridge with dual-mode operation and coloured console tracing
Agent Layer	Integrated OpenRouter LLM with 12+ tools and ChromaDB persistent memory
Web Interface	Flask app with user auth, session persistence, multi-format file upload, and CORS

Testing	Stress-tested concurrent process creation, page eviction under all four policies, semaphore blocking, and deadlock scenarios
Integration	End-to-end verification: browser chat → Flask → agent → kernel bridge → C++ kernel

5. Project Implementation

5.1 Multi-threaded Kernel Architecture (kernel/kernel_server.cpp)

The main thread listens on TCP port 9000, accepts connections, and dispatches each JSON syscall packet to the appropriate handler. Three additional kernel threads (scheduler_tick, memory_monitor, stats_printer) run concurrently, all coordinated through atomic flags. The kernel boots by initialising the process table, page table (policy selected via command-line flag), semaphore manager, and scheduler.

5.2 Process Control Block — Key Code

```
// Effective priority prevents starvation int effective_priority() const { int boost =
aging_ticks / 5; return std::max(0, priority - boost); } // State transition with timestamp
update bool transition(int pid, ProcessState new_state) { std::lock_guard<std::mutex>
lock(mtx_); ... it->second.state = new_state; it->second.last_active_at = _now_ms(); return
true; }
```

5.3 Page Replacement — CLOCK Algorithm

```
case ReplacementPolicy::CLOCK: { int n = (int)ring_.size(), checked = 0; while (checked < 2*n) {
int pid = ring_[clock_hand_]; if (!table_[pid].ref_bit) { victim = pid; clock_hand_++; break; }
table_[pid].ref_bit = false; // give second chance clock_hand_++; checked++; } } break;
```

5.4 Deadlock Detection — DFS Cycle Check

```
// Build waits-for graph from assignment + request edges for (auto& [pid, res_set] :
request_edges_) { for (auto& res : res_set) { for (int holder : assignment_edges_[res])
waits_for[pid].insert(holder); } } // DFS on waits-for graph — cycle = deadlock if (_dfs(pid,
waits_for, visited, rec_stack, cycle)) { std::cout << "[DEADLOCK DETECTED] Cycle: ...";
std::cout << "[Deadlock] Resolution: preempt PID=" << cycle.back(); }
```

5.5 System Overview — Component Interaction

Component	Role
Browser / Client	User chat, file upload, authentication
Flask Web App	Session management, REST endpoints, file processing
LLM Agent	ReAct loop, tool calls, ChromaDB memory, OpenRouter API
Kernel Bridge	Python-to-C++ syscall proxy; Python simulation fallback
C++ Kernel Server	TCP listener on port 9000; syscall dispatch
ProcessTable	Mutex-protected PCB store; ready-queue sorted by eff. priority
PageTable	RAM frames + LRU/FIFO/CLOCK/LFU eviction; VectorDB overflow
SemaphoreManager	Named semaphores, FIFO wait queues, deadlock detection

Scheduler (3 threads)	Tick / memory monitor / stats printer running concurrently

6. Results

The system was tested under concurrent multi-user scenarios and deliberate stress conditions for each OS subsystem.

Outcome	Detail
Process Lifecycle	Fork/run/wait/exit transitions verified; PCB state machine enforced correctly
Priority Scheduling	Aging ticks confirmed to boost long-waiting processes; no starvation observed over 300-tick runs
Page Replacement (LRU)	All 10 frames filled; LRU correctly evicted the least-recently accessed page under mixed read/write workloads
Page Replacement (CLOCK)	Reference bits cleared and set as expected; second-chance semantics confirmed through trace logs
Deadlock Detection	Circular wait scenario with 3 processes and 2 semaphores detected and reported in under 1 ms
Syscall Interface	All 14 syscalls tested via both live C++ kernel and Python simulation; JSON responses validated
Simulation Fallback	Full functionality confirmed without C++ binary; Python bridge emulated scheduling, paging, and semaphores accurately
Web Interface	Multi-user login, chat history isolation, and 50 MB file upload functional under concurrent sessions

7. Conclusion

LLM-OS v3 successfully demonstrates the practical implementation of core Operating Systems concepts in a realistic, multi-layered software system. The C++ kernel provides production-quality process, memory, scheduling, and synchronisation subsystems. The Python bridge and web interface expose these capabilities to end-users without sacrificing the educational integrity of the OS design.

The project uniquely bridges classical OS theory and modern AI by pairing a kernel implementing textbook algorithms (LRU, CLOCK, priority aging, deadlock detection via resource-allocation graphs) with a language-model agent that can reason about system state, remember past interactions, and call external APIs. This combination demonstrates that OS abstractions remain relevant and extensible in an AI-driven computing landscape.

Future enhancements could include a graphical kernel visualiser, on-demand policy switching through the web UI, distributed process migration across nodes, and reinforcement-learning-based dynamic scheduling.

8. OS Concepts Demonstrated

Concept	Implementation
Process Management	PCB state machine: NEW → READY → RUNNING → WAITING → TERMINATED
Process Scheduling	Priority + Round-Robin with configurable 500 ms quantum

Starvation Prevention	Aging ticks auto-boost: effective_priority = priority - aging/5
Virtual Memory	Page table with physical-virtual frame mapping
Page Replacement	LRU (linked list), FIFO (queue), CLOCK (ref bits), LFU (frequency counter)
Memory Eviction	Automatic swap to VectorDB secondary storage on frame overflow
Synchronization	Named binary + counting semaphores with FIFO wait queues
Deadlock Detection	Resource-allocation graph DFS cycle detection on every blocking wait()
System Calls	14 syscalls: process, memory, semaphore, file I/O categories
IPC	TCP socket JSON-RPC on port 9000 between Python bridge and C++ kernel
Multi-threading	4 kernel threads: main acceptor + scheduler tick + memory monitor + stats
Thread Safety	std::mutex guards on ProcessTable, PageTable, SemaphoreManager, VectorDB

9. Project Statistics

Metric	Value
Total Lines of Code	~3,811 (C++ kernel: ~1,382 | Python: ~2,429)
C++ Kernel Files	7 (pcb.h, page_table.h, scheduler.h, semaphore.h, syscall.h, vector_db.h, kernel_server.cpp)
Python Files	3 (agent.py, kernel_bridge.py, run.py)
System Calls Implemented	14
Page Replacement Policies	4 (LRU, FIFO, CLOCK, LFU)
Background Kernel Threads	3 (scheduler_tick, memory_monitor, stats_printer)
Synchronization Primitives	2 types (binary semaphore, counting semaphore)
LLM Agent Tools	12+
Supported File Formats	25+
Default RAM Capacity	10 frames (configurable)
Default Time Quantum	500 ms (configurable)

10. References

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- GNU Compiler Collection (GCC) Documentation. Retrieved from https://gcc.gnu.org/
- Linux Manual Pages. POSIX Threads Programming Documentation. Retrieved from https://man7.org/linux/man-pages/
- The Open Group. POSIX Standard Documentation. Retrieved from https://pubs.opengroup.org/
- FAST-NUCES Department of Computer Science. Operating Systems Course Notes and Lab Manuals.
- Ubuntu Linux Documentation. Retrieved from https://ubuntu.com/server/docs
- OpenRouter API Documentation. Retrieved from https://openrouter.ai/docs

• ChromaDB Documentation. Retrieved from https://docs.trychroma.com/